

Full Length Article

MoEP: Compact and efficient sparsity with modular expert paths

Joonas Tapaninaho , Mourad Oussalah *

University of Oulu, Faculty of ITEE, CMVS, Pentti Kaiteran Katu 1, Oulu, 90570, Finland

ARTICLE INFO

Keywords:

Language models
Mixture-of-experts
Modular architecture

ABSTRACT

The transition from dense to sparse model architectures has become a key trend in the field of Large Language Models (LLMs).

Mixture-of-Experts (MoE) methods can be used to increase model conditional representation capacity by activating only a subset of parameters for each input token. However, the practical efficiency of such sparsity depends on the routing design and hardware implementation. This paper introduces MoEP (Modular Expert Paths), a layer-level routing architecture, which is designed to study sparsity under a fixed and comparable parameter budget. MoEP parallelizes decoder-only Transformer blocks with MoE-style linear projections to implement selective token-dependent routing across reduced-dimensional paths. Originally MoEP was developed in the BabyLM setting using GPT-2 as its architectural baseline. We extend this study by applying the same MoEP routing and overall design to additional small decoder-only baseline architectures. To further examine architectural transferability, we applied the MoEP structure to GPT-2-, LLaMA-, and Gemma-style small baselines trained under the same protocol. Each one of those baselines was converted into a MoEP variant using the same modular routing design, enabling controlled comparison across architectures.

All models were trained and evaluated under the official BabyLM strict-small protocol to ensure comparability.

The results show small average gains with stronger task-specific improvements, but also task-dependent decrease, which indicates that MoEP's benefits are architecture and scale dependent. A larger scale comparison using the Pythia-1B model as baseline, further suggests that the current layer-level MoEP design probably does not inherit the same scalability and efficiency advantages as standard FFN-level MoE. This is because it routes complete Attention-FFN blocks rather than only feed-forward experts.

1. Introduction

Recent open Large Language Models (LLMs), including GPT-OSS (OpenAI, 2025), Llama 4 (Meta-AI, 2025),¹ (DeepSeek-AI, 2025), and Mixtral (Jiang, 2024), illustrate a growing trend toward sparse and routed computation, where representational capacity is increased while the number of active parameters per token is controlled. Unlike dense models, where each token follows the same computation path, MoE-style models route tokens through a subset of available experts rather than activating all parameters for every token.

Prior to LLMs and recent transformer architectures, MoEs were originally proposed to enhance the training performance of neural network architectures using several expert networks trained on different partitions of the input space as in earlier work of Chen et al. (1999), Jordan and Xu (1995). The latter analyzed the stability of the MoE architectures when trained using Expected-Maximization (EM) and gradient-based

training algorithms and their convergence properties, especially when dealing with multiclass classification problems. Despite some progress, ensuring full-scale training stability remains an open challenge and no universally optimal solution emerged. In our setting, we leverage *auxiliary loss* during training to maximize expert network utilization, and, thereby, enhance training stability.

Classical MoE models were originally introduced in the context of small neural networks and hierarchical mixtures (Jacobs et al., 1991; Jordan & Jacobs, 1994), while later models adopted exponential-family regression models (Jiang & Tanner, 1999). In these foundational models, routing is probabilistic and dense, where all experts are evaluated and combined for each input. Although this can be manageable on a smaller scale, evaluating every expert becomes unreasonably expensive for modern large models. As a result, modern MoE architectures implement sparse routing, where only a subset of model parameters is activated at the token level for each forward pass. Although sparsity

* Corresponding author.

E-mail addresses: Joonas.Tapaninaho@oulu.fi (J. Tapaninaho), Mourad.Oussalah@oulu.fi (M. Oussalah).¹ At the time of writing this paper, no peer-reviewed publication exists for Llama 4, which is why we cite the official announcement. More background and detailed information on the Llama family of models can be found in Touvron (2023a) and Touvron (2023b)

enables scaling of representational capacity at roughly fixed compute scale, it also requires managing a substantially larger total parameter count.

Most scalable MoE language models apply routing at the feed-forward network level, while keeping attention shared across tokens. This distinction is important because FFN experts can be sparsely activated more directly than full Transformer blocks that include self-attention. Recent and previous work has examined *sparse and routing-based models* (DeepSeek-AI, 2024; Du et al., 2022; Jiang, 2024; Muenighoff, 2025; OpenAI, 2025) and some of them have also explored alternative sparsity approaches that aim to improve efficiency and rethink conditional computation.

Mixture of Recursions (MoRs) (Bae et al., 2025) used depth-based sparsity, where different tokens communicate with different depth stacks of layers to achieve efficiency. On the other hand, the compositional approach *PaPaformer* (Tapaninaho & Oussalah, 2025) introduced a method to remodel Transformer layers into smaller *parallel sub-paths*, which can be used as independent trainable modules.

These efforts highlight a broader trend to improve efficiency and flexibility by enabling tokens to follow different computation paths.

This paper extends a preliminary MoEP research presented at the First BabyLM Workshop (Tapaninaho, 2025),

which introduced *MoEP* (Modular Expert Paths) to study layer-level routing by combining two routing mechanisms

within a decoder-only language model: (i) *top-k* token routing across *Parallel Transformer* blocks, and (ii) *Mixture-of-Experts* layers based on lightweight linear projections. As a result, each token activates only a limited set of parallel blocks and experts in a forward pass, creating more diverse computational pathways under a fixed or comparable parameter budget. We extend the MoEP idea from a single GPT-2 baseline to a general framework applied to GPT-2, Llama, and Gemma architectures.

The MoEP-SwiGLU variant mentioned in the original workshop version is not included in our experiments in this paper because it underperformed in comparison to more simpler linear-projection. Therefore, this paper focuses on more stronger linear MoEP design and their transferability across different baseline architectures.

In addition, we provide a more detailed analysis of expert and layer-level routing behavior, expand the quantitative evaluation, and include an extended discussion of related work.

We trained all our MoEP-based models using the *BabyLM strict-small track*² protocol and the official evaluation pipeline. Initially, MoEP was developed using GPT-2 as a baseline architecture.

To examine architecture-dependent behavior, we applied the same MoEP structure to *Llama* (Grattafiori, 2024; Touvron, 2023a,b) and *Gemma* (Team, 2024a,b) to create *MoEP-Llama* and *MoEP-Gemma* variants. All our models were trained and evaluated under identical BabyLM strict-small scale conditions.

We use GPT-2 as the initial layer-wise baseline architecture because it aligns with the official BabyLM GPT-2 baseline and that way provides a simple controlled reference point.

In addition to GPT-2, BabyLM also provides GPT-BERT (Charpentier & Samuel, 2024) as an alternative baseline, which combines causal and masked language modeling into a masked next-token prediction objective.

Under the official BabyLM evaluation protocol, MoEP-GPT-2 outperformed all BabyLM strict-small baseline models, including both GPT-2 and GPT-BERT.

Our extended architecture comparisons show small average improvements for all MoEP variants at certain scale, but also substantial task-dependent variation and decreases at larger scale in current setup.

We also showed that improving the routing mechanism increases the performance within parallel architecture even though MoEP did not employ the *PaPaformer* (Tapaninaho & Oussalah, 2025) style of modu-

larity, in which independent modules are partly pre-trained separately, which prior work suggests as the major component of the performance increase.

Overall, the main contributions of this work are summarized below:

1. We propose *MoEP*, a modular sparse decoder-only architecture that integrates layer-level expert networking across parallel blocks.
2. We provide a BabyLM-compliant evaluation on the strict-small track, comparing MoEP against dense baseline models under matched conditions.
3. We extend the same MoEP design to GPT-2-, LLaMA-, and Gemma-style small baselines to examine architecture-dependent behavior.
4. We analyze expert and layer-level routing behavior and show that routing balance varies substantially across architectures and tasks.
5. We further analyze MoEP scalability at the one billion parameter scale and show that, within the current setup, it cannot improve the evaluation score of the dense baseline.

2. Related works

2.1. Small scale language models

Boosting the performance of language models has increased interest in small-scale rapid experimentation. This growing line of work investigates how to train useful language models and evaluate them under strict data and parameter budgets. Instead of scaling to hundreds of billions of tokens, these approaches emphasize sample-efficient pretraining on small but effective corpora, and often target architectures in the tens of millions of parameters. The BabyLM framework (Charpentier et al., 2025) explicitly focuses on this regime and provides carefully curated small-scale corpora and standardized evaluation protocols for models such as GPT-BERT (Charpentier & Samuel, 2024).

Beyond BabyLM, many smaller transformer variants, for example, distillation or compression-based models reduce model size but typically at the cost of reduced performance. However, most of these approaches still rely on dense architectures and have not explored sparse expert routing in low-resource pretraining settings. Our work complements this line of research by investigating a layer-level MoE design under the BabyLM strict-small constraints, aiming to increase effective representational capacity without increasing the total number of parameters.

2.2. Parallel architectures

As an alternative to dense and sparse expert styles of architecture design, another research trend focused on parallelization to increase expressiveness or efficiency. Despite its promising results, this trend has received little attention in the LLM research community. PaLM (Chowdhery, 2023) uses parallelization within layers to achieve faster training with only minor quality degradation, while Branchformer (Peng, 2022) combines attention and multi-layer-perceptron (MLP) with convolutional gating into parallel components to capture both local and global context. More recently, Papaformer (Tapaninaho & Oussalah, 2025) proposed an alternative approach in which independently trained parallel paths are combined into a larger composite model. The aim of this work is to achieve efficiency by scaling down dimensionality across parallel layers, while maintaining performance through the increased representation enabled by parallelism. MoEP builds on this idea by maintaining parallelism but coupling it with sparse MoE-style top-k routing.

2.3. Mixture-of-experts

2.3.1. Dense vs. sparse expert architectures

After successful adaptation of MoE into Transformer-based language models (Vaswani et al., 2017), sparse expert architectures have become

² <https://babylm.github.io>.

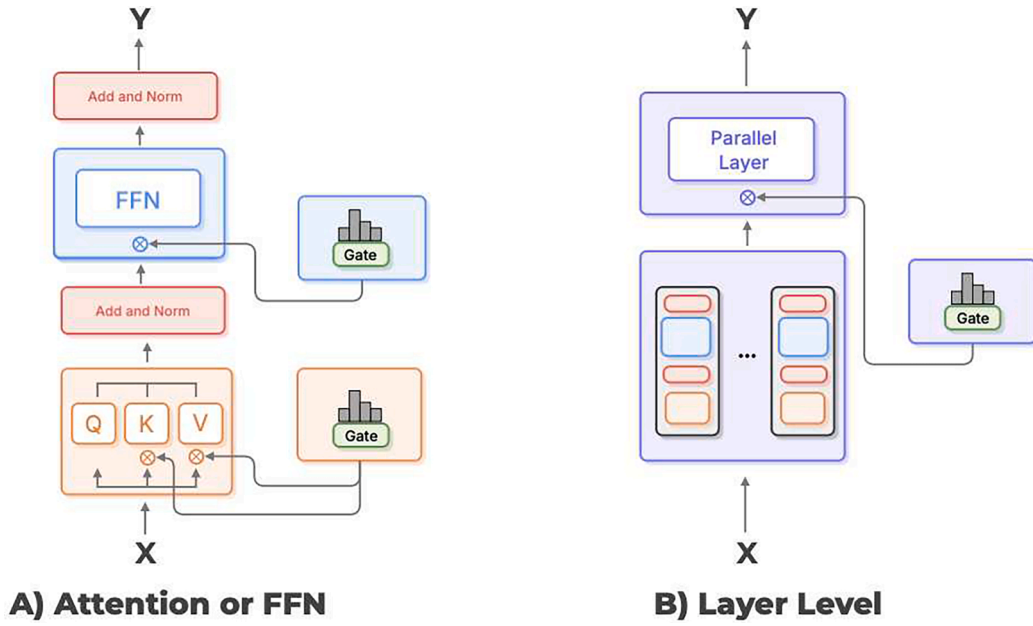


Fig. 1. Expert-placement strategies in transformer architectures. **A:** Standard sublayer-level MoE places experts inside the attention module, the FFN module, or both, while the overall Transformer layer structure remains shared. **B:** MoEP applies routing at the layer level: A parallel-layer router selects the top- k paths among P complete parallel Transformer blocks with identical structure but distinct parameters, and then aggregates their outputs.

almost standard for increasing representation power without adding computational density. This often means that the model is directly pre-trained using a sparse network of experts in some part of the architecture. However, there are several works such as Dou et al. (2024), Wu et al. (2024), Zhu et al. (2023), where the pre-trained model itself is dense, but in the fine-tuning phase sparsity is added using methods like Low-Rank Adaptation (LoRa) (Hu et al., 2022). These models benefit from stability in the pre-training phase, where all model parameters are activated and tuned at each iteration, while in the fine-tuning phase switching to sparsity enables stronger specialization and increased representation power. On the other hand, there is much variation in sparsity strategies among models that add sparsity after dense pre-training. For example, in fine-tuning phase, *MoLe* (Wu et al., 2024) uses LoRa weights at the layer level for a pre-trained and frozen model, while *LoraMoE* (Dou et al., 2024) applies LoRa weights only at the Feedforward network (FFN) level. *DS-MoE* (Pan et al., 2024) takes a different approach, where expert networks are trained as dense but, during inference, the model switches to sparse activation to achieve efficiency.

Among sparse expert architectures, we distinguish representations such as *THOR* (Zuo et al., 2022) that randomly selects experts during training, while in *MoE-ECR* (Zhou, 2022), unlike conventional MoE architectures where tokens select top-scoring experts, each expert selects top- k tokens (reversal operation).

The next sections explore different placement strategies for experts, as illustrated in Fig. 1 and summarized in Table 1.

2.3.2. FFN-level MoE

Since feed-forward networks dominated the parameter count in Transformer models, replacing the FeedForward sublayer with expert networks is a popular approach to achieve efficient MoE. Many previous works have explored different methods to optimize efficiency and improve performance of such an approach when dealing with large scale networks. *Switch Transformers* (Fedus et al., 2022) used top-1 routing among experts and achieved significant increases in pre-training speed. *LoRAMoE* (Dou et al., 2024) froze pre-trained model weights and used LoRa to create expert networks, while *AdaMix* (Wang, 2022) advocated adapter-based expert networks after the FFN. State-of-the-art models such as *DeepSeek-AI* (2025) and *Llama-4* (Meta-AI, 2025) employed both routed and shared experts, whereas *GPT-OSS* (OpenAI, 2025) and *Mistral* (Jiang, 2024) promoted routed expert-based mechanism.

2.3.3. Attention-level MoE

Typically attention-level experts operate on key and value projections, enabling specialized attention heads that capture diverse relational patterns. This leads to an increase in routing complexity within the attention mechanism in comparison to FFN-level routing. Most of the works that used this approach are related to parameter-efficient fine-tuning (PEFT). For instance, *SiRA* (Zhu et al., 2023) used LoRa for query and key weights in attention, while the pre-trained model is kept frozen. *MoA* (Fu, 2024), on the other hand, does not use trainable weights, but heterogeneous elastic rules for sparse pattern selection.

2.3.4. Attention + FFN

Models such as *JetMoE* (Shen et al., 2024) that use expert network concurrently at both Attention and FFN levels are found to outperform Llama and DeepSeek models of comparable size. *Mov* (Zadouri et al., 2024) introduces an interesting approach, where instead of using LoRa directly to model weights, it employs a mixture-of-vectors that are added to the intermediate results in Attention and FFN sublayers.

2.3.5. Layer-level MoE

Instead of replacing the Attention or FFN sublayer, layer-level MoE uses parallel layers as experts as illustrated in Fig. 1. This approach provides flexible and sparse pathway selection at the block level. Layer-level expert networks remain a relatively unexplored area, with the exception of *MoLe* (Wu et al., 2024), which applies LoRa weights at the layer level to a frozen pre-trained model.

The intention of this work is to address this gap with a novel approach, where parallel expert layers operate at smaller dimensionality. In this way, the overall parameter size is kept fixed while model sparsity is increased.

2.4. MoE efficiency and compression

Recent work has also emphasized that MoE efficiency cannot be only evaluated by model active parameter count. Zhang et al. (2024) showed that in MoE training, activations, memory usage and communication overhead can be major bottlenecks. In the case of MoEP, this is especially relevant, because its sparsity is applied at the level of complete layer blocks rather than only at the FFN expert level. Because of that, even when the number of active model parameters is reduced, attention

Table 1

Overview of representative mixture-of-experts (MoE), routing-based, and parameter-efficient fine-tuning (PEFT) architectures.

Model Name	Key Characteristics	Training setting	MoE placement	Limitations
MoLE	Applies LoRA weights at the layer level to a frozen pre-trained model	PEFT on frozen pretrained dense model	Layer Level	Requires frozen base model and benefits depend heavily on it
LoRA MoE	Uses LoRA weights as expert networks, which are added into shared weights	PEFT on frozen pretrained dense model	Feed-Forward Network	Experts are restricted to FFN LoRA adapters, and the frozen backbone limits representational change
DS-MoE	Dense training, where all experts are activated. During inference, model changes to sparse expert routing	Dense pretraining with sparse inference	Feed-Forward Network + Attention	Dense experts training is costly, while sparse-only inference can lead train–test mismatch and hurt robustness
Thor	Randomly selects experts during training	Sparse pretraining	Feed-Forward Network	Relies on stochastic expert selection during training, which prevents routing from becoming fully task-adaptive
MoE-ECR	Reverses standard MoE routing. Instead of each token selecting top-k experts, each expert selects top-k tokens to process	Sparse pretraining	Feed-Forward Network	Expert-side token selection, instead of token-side selection, complicates routing decisions and load balancing across experts
AdaMix	Uses adapter based expert networks after feed-forward network, which are randomly selected during training and average all experts during inference	PEFT on frozen pretrained dense model	Feed-Forward Network	Random expert selection during training but experts averaged at inference limiting expert specialization
SiRA	Similar to AdaMix, but instead of randomly selecting experts, it uses auxiliary loss and dropout to train balanced router	PEFT on frozen pretrained dense model	Attention	Expert adapters on a frozen backbone require carefully tuned auxiliary loss and dropout
MoA	Does not use trainable weights, instead it uses heterogeneous elastic rules for sparse pattern selection	Inference-time attention-pattern selection	Attention	Effectiveness depends on hand-crafted rules rather than learned routing
JetMoE	Uses expert network concurrently at both Attention and FFN levels and outperforms Llama and DeepSeek model of comparable size	Sparse pretraining	Feed-Forward Network + Attention	MoE in both FFN and attention adds complexity and raises total parameters relative to FFN- or attention-only MoE
Mov	Instead of using LoRA directly to model weights, it employs a mixture-of-vectors that are added to the intermediate results in Attention and FFN sublayers	PEFT on frozen pretrained dense model	Feed-Forward Network + Attention	Uses low-capacity vector experts on a frozen model, which limits overall effectiveness and expressivity
MoEP	Uses parallel expert layers with smaller hidden dimensions. MoE-style shrink and grow projections encapsulate parallel layers and adjust hidden dimensionality	Sparse pretraining from scratch	Layer Level	Validated only in small-data and added routing and parallel blocks increase architectural complexity

computation and routing overhead may limit the practical wall-clock and memory benefit.

Another related line of work studies compression and removal methods as simpler methods to provide efficiency gains without introducing additional routing complexity. [He et al. \(2025\)](#) investigated several compression techniques for MoE models, including Layer Drop and Block Drop, and showed that much of the original model performance can be preserved after removing redundant components. These approaches differ from MoEP because they reduce or remove existing components rather than introducing additional routing during pre-training. MoEP, in contrast, directly modifies the architecture by introducing conditional layer-level routing through reduced-dimensional parallel paths. Despite the discrepancy in strategies, compression-based methods pro-

vide a complementary aspect to MoEP. In particular, MoEP should not be interpreted as an automatically preferable option to simpler compression approaches, because its advantage depends on whether conditional layer-level routing provides sufficient benefits to justify the additional architectural complexity.

3. Methods

3.1. Model architecture

Overview

Our MoEP baseline architecture (see [Fig. 2](#)) interleaves two standard GPT-2, Gemma or Llama (dense) layers with a sparse middle stack: *Layer*

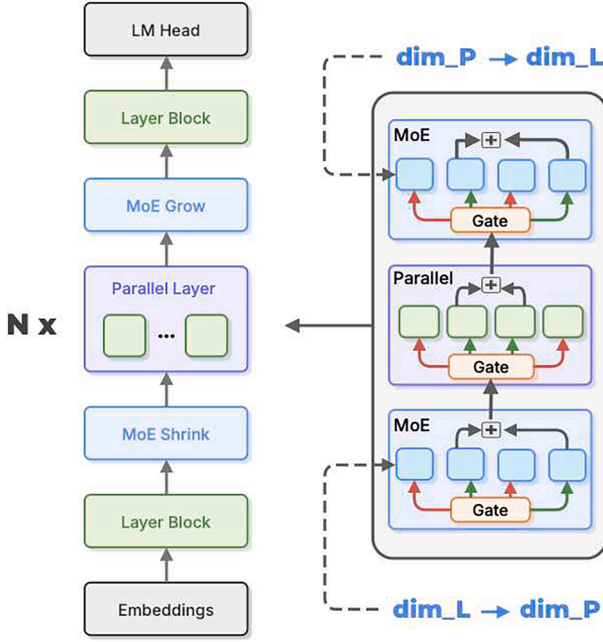


Fig. 2. MoEP architecture overview. The model begins and ends with standard full-dimensional Transformer layer blocks. Between them, MoE shrink and grow projections map the hidden representation between the large model dimension d_L and the smaller parallel-path dimension d_P . The central module contains N routed parallel layers, where each parallel layer selects a subset of paths and aggregates their outputs. Thus, MoEP applies conditional routing at the level of complete reduced-dimensional Transformer blocks rather than only within the FFN sublayer.

(full size) $\rightarrow$ MoE Block (shrink) $\rightarrow$ Parallel Layer (repeated N times with top- k gating) $\rightarrow$ MoE Block (grow) $\rightarrow$ Layer (full size). Both the initial and terminal layers operate at a higher hidden dimension dim_L . The first MoE Block uses E experts with top- k gating to map projection into smaller hidden dimension dim_P suited for the parallel stack (see Fig. 2). Intermediate Parallel Layer uses top- k gating among P Layers, which operates at a corresponding smaller hidden dimension dim_P . After the N Parallel Layer, the second MoE Block performs the inverse procedure, mapping the projection from dim_P back to dim_L before the terminal layer.

In the following sections, we use MoEP as a common term for the proposed sparse framework. The dense baseline architectures Llama, Gemma, and GPT-2, which are converted to sparse variants using MoEP, are referred to as MoEP-Llama, MoEP-Gemma, and MoEP-GPT-2, respectively. These variants follow identical principles while retaining each baseline model's native layer configuration (see Table 2). Due to the small size of our models, we employ Multi-Head Attention (MHA) for all of them instead of other lightweight mechanisms such as Grouped-Query Attention (used in Gemma-2 and Llama-3), which primarily benefit larger models.

MoE block projections (Shrink and grow)

When alternating between two hidden dimensionalities dim_L and dim_P , MoE Blocks ensure a smooth transition and avoid a possible information bottleneck, where overly rapid dimensionality changes could potentially discard useful information. In this regard, we implemented two MoE Block projections using the following dimensionality transition:

$$dim_L \rightarrow \text{shrink} \quad dim_P \rightarrow \text{grow} \quad dim_L \quad (1)$$

Both MoE Blocks consist of E experts with token-level top- k gating over experts. In our MoEP model variants, MoE Block experts are simple linear projections.

Table 2

Architectural hyperparameters of the GPT-2-, LLaMA-, Gemma-, and MoEP-style small decoder baselines. All models are trained from scratch under the BabyLM strict-small protocol and are not pre-trained, instruction-tuned, or command-tuned checkpoints. In MoEP models, shared parameters are reused across the routed structure, and entries marked with * are determined by the corresponding dense baseline configuration.

Hyperparameter	GPT-2	Llama	Gemma	MoEP
Vocabulary size	$\sim 16K$	$\sim 16K$	$\sim 16K$	$\sim 16K$
d_{model}	384	384	384	384 / 192
Layers	12	12	12	14
Parallel blocks	–	–	–	4
Heads	6	6	6	6 / 3
Head dimension	64	64	64	64
FF multiplier	4	~ 2.67	4	*
FF type	MLP	SwiGLU	GELU	*
MoE FF type	–	–	–	Linear
Positional Embeddings	APE	RoPE	RoPE	*
Sliding Window	–	–	256	*
N experts	–	–	–	4
Top k	–	–	–	2
Normalization	LN	RMS	RMS	*
Attention	MHA	MHA	MHA	MHA
Train seq len	512	512	512	512
Total Parameter (millions)	28M	34M	34M	28 - 35M
active				19 - 23M

Parallel layers

Each Parallel Layer contains P Parallel blocks $\{B_1, \dots, B_K\}$, which are architecturally equivalent with Layer Block, but operate at a reduced dimension dim_P . The Parallel Blocks share identical sublayer structure but have fully disjoint parameters. Linear router is shaped $dim_P \times P$ and it applies a token-level top- k selection among the P Parallel Block, where the routed inputs are summed together. This routing method implements a layer-level expert networking, allowing different tokens to traverse different subsets of Parallel blocks (e.g. layers) within each Parallel Layer. Stacking N Parallel Layers yields a deep routed path in compact dimensions.

Routing objective and training loss

To avoid expert (MoE) and Parallel block (Parallel Layer) collapse, which is a typical problem in sparse expert networks, we use the standard load-balancing regularizer in the training phase. More specifically, let p_i denote the average routing probability assigned to Parallel block or expert i over a batch. The balancing term is defined as:

$$\mathcal{L}_{\text{balance}} = - \sum_i p_i \log p_i \quad (2)$$

computed separately for Parallel block routing and expert routing. The total objective is then:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda^{\text{block}} \mathcal{L}_{\text{balance}}^{\text{block}} + \lambda^{\text{expert}} \mathcal{L}_{\text{balance}}^{\text{expert}} \quad (3)$$

where $\mathcal{L}_{\text{CE}}$ is the next-token cross-entropy loss and λ is the learning weight.

3.2. Training setup

To explore our idea in low resource setup, we follow official BabyLM (Charpentier et al., 2025) framework for model pre-training and evaluation. This setup provides a benchmark for small-scale language models and enables direct comparability with BabyLM baseline models.

3.2.1. Dataset and tokenization

To pre-train our models, we used the BabyLM strict-small training corpus, which contains a little over 10 million words from curated English sources. The BabyLM corpus is designed especially for small-scale

Table 3
Training setup and optimization parameters.

Hyperparameter	Value
Optimizer	AdamW
Learning rate	3×10^{-4}
Batch size	16
Training epochs	10
Gradient accumulation steps	1
Weight decay	0.1
Adam betas	(0.9, 0.95)
Adam epsilon	1×10^{-8}
Scheduler type	Cosine
Warmup steps	800
Random seed	42

efficient and fast training, which helps to achieve novel experimentation with low computational and memory resources. To maintain compatibility with *BabyLM* baselines and our previous work, we trained a *byte-pair encoding* (BPE) tokenizer with a 16K fixed vocabulary and a pattern-recognition strategy similar to that of the baseline GPT-2,³ ensuring maximum similarity between models. All our models share that same tokenizer and are trained using the same *BabyLM* strict-small training corpus.

3.2.2. Training procedure

To ensure a controlled architectural comparison, we trained six models in total, containing three dense baselines (GPT-2, Llama, and Gemma) with the corresponding sparse variants (MoEP-GPT-2, MoEP-Llama, and MoEP-Gemma), under identical *BabyLM* strict-small conditions.

The specifications for all models are shown in Table 2, where the GPT-2 configuration corresponds to the official *BabyLM* baseline and the other model configurations mirror the same small-model setup. All models are trained from scratch under the *BabyLM* strict-small protocol and are not officially pretrained or instruction-tuned checkpoints.

To achieve stable model training, we used the AdamW optimizer with a cosine learning rate decay, combined with standard dropout and weight decay regularization (see Table 3).

3.2.3. Training process

The training data were pre-tokenized using a *stride of 128* and during training, examples were randomly sampled from the full pre-tokenized dataset using an epoch-based shared seed. This ensures that all models were trained on identical input samples. Our pre-training followed the *BabyLM* strict-small protocol, where each model was trained for 10 epochs, and each epoch was terminated after approximately 10 million words had been processed. During pre-training, checkpoints were saved every 1 million words up to 9 million words, and subsequently every 10 million words up to 100 million words. We performed fast evaluations on all checkpoints, and the final model weights used for full evaluation and further analysis were selected from the checkpoint that achieved the best validation performance.

This procedure did reveal notable differences in model learning efficiency, which are discussed in more detail in Section 4.2.

3.2.4. Implementation environment

We conducted all our experiments using a single NVIDIA A100 GPU in CSC's Puhti supercomputing environment (CSC, 0000). Training a single model with 10 epochs required approximately 1-2 h. This duration could be further reduced with code optimizations.

The implementation is performed using PyTorch and Hugging Face libraries and released for reproducibility⁴ and the model is directly downloadable from Hugging Face.⁵

³ <https://huggingface.co/BabyLM-community/babyLM-baseline-10m-gpt2>.

⁴ <https://github.com>.

⁵ <https://huggingface.co/Jtapsa>.

3.3. Evaluation protocol

All evaluations followed the official *BabyLM* pipeline (Charpentier et al., 2025), which comprises both zero-shot and fine-tuning benchmarks for assessing linguistic and reasoning capabilities. The evaluation suite includes:

- **Zero-shot tasks:** BLiMP, EWOK, Entity Tracking, WUG Test, Compositionality, Reading, and AoA (Age of Acquisition).
- **Fine-tuning tasks:** BoolQ, MNLI, MRPC, MultiRC, QQP, RTE, and WSC.

For fine-tuning tasks, the *BabyLM* pipeline provides task-specific filtered training data and fine-tuning parameters, which were adopted directly to ensure comparability.

4. Results

4.1. Quantitative results

Table 4 shows the evaluation results on the *BabyLM* benchmark tasks for our primary model (GPT-2-MoEP) and for the official *BabyLM* baselines. Across all tasks, our MoEP achieves the highest overall performance, surpassing its dense GPT-2 baseline and the other GPT-BERT variants under the strict-small track. This is obtained thanks to the AoA task, where it outperformed all other models. Even when excluding AoA from the macro average, its average performance remains higher than the dense GPT-2 baseline, which serves as our closest architectural comparison point, while it struggled slightly to match the GPT-BERT hybrid models.

In particular, MoEP also achieved the best scores in five individual tasks, which is more than any other evaluated model in the *BabyLM* baseline comparison. This demonstrates that using modular sparse routing as a framework for dense baselines can yield competitive results even in the limited-data regime and small-model setup.

To provide a broader comparison for MoEP as a framework, we experimented with the same setup using more architectures. Table 5 presents the results for six state-of-the-art models, where the dense baseline models are (GPT-2, LLaMA, Gemma) and their corresponding MoEP variants. In the table, %-change represents the improvement or degradation between each dense baseline model and its sparse MoEP variant across both zero-shot and fine-tuned tasks.

This broader comparison revealed several interesting phenomena. For example, MoEP-GPT-2 exhibits the largest absolute improvement among all model pairs (+170% in Entity Tracking), while MoEP-LLaMA shows the largest absolute degradation (-35.30% in Entity Tracking).

Those large task-level changes show that MoEP does not provide uniform task-level improvements, which in some cases are strongly architecture- and task dependent.

MoEP-GPT-2 shows minor degradations across most of the tasks, but a single large improvement leads to an overall performance gain. In the case of MoEP-LLaMA, improvements were observed in most fine-tuning tasks, where strong gains in RTE and WSC helped to offset the large degradation in Entity Tracking and resulting in a positive average change. MoEP-Gemma exhibits several large performance shifts in both directions, indicating an architectural tendency to specialize across different tasks. Similar to MoEP-GPT-2, a substantial improvement in Entity Tracking contributes to a positive overall change.

Overall, the results indicate that MoEP variants achieve small average improvements over their dense baselines, but these gains are not uniform across tasks and some tasks show substantial improvements, while others show clear degradation. This suggests that MoEP should not be interpreted as consistently superior to the corresponding dense architectures. Instead, the effectiveness of MoEP depends on the underlying baseline architecture, which influences the routing mechanism and capacity of the routed layers. Tasks such as Entity Tracking show

Table 4

Evaluation scores on BabyLM tasks for our baseline MoEP model (top) and hugging face baseline models (bottom). In the table, **BLiMP** refers to the average over BLiMP and BLiMP-supplement, **WUG** corresponds to the average of wug adjacency and wug past tense, and **Readings** is the average of eye tracking and self-paced reading tasks.

Model	Zero-shot Tasks							Finetuned Tasks							Macro
	BLiMP	EWOK	Entity	WUG	Comps	Reading	AoA	BoolQ	MNLI	MRPC	MultiRC	QQP	RTE	WSC	Avg
Our Models															
GPT-2-MoEP ^a	59.15	50.20	35.65	33.00	50.70	6.70	53.70	66.20	48.10	70.10	64.50	70.75	62.60	67.30	44.50
HF Baselines															
GPT-2 ^b	61.75	49.90	13.90	30.55	51.70	6.50	11.7	52.10	33.10	67.60	57.50	63.60	56.10	61.50	37.40
GPT-BERT (causal) ^c	67.45	49.50	34.60	36.05	52.80	6.70	-3.90	68.10	46.90	74.50	68.30	76.70	56.10	65.40	41.20
GPT-BERT (focus-causal) ^d	62.35	49.5	31.10	32.70	52.90	6.50	3.8	67.60	51.80	78.90	67.40	77.40	57.60	61.50	40.00
GPT-BERT (mixed-causal) ^e	65.60	50.20	25.40	48.50	25.00	6.40	14.50	66.70	53.30	77.50	67.00	76.60	55.40	63.50	39.20

^a <https://huggingface.co/Jtapsa/moep>

^b <https://huggingface.co/BabyLM-community/babylm-baseline-10m-gpt2>

^c <https://huggingface.co/BabyLM-community/babylm-baseline-10m-gpt-bert-causal-focus>

^d <https://huggingface.co/BabyLM-community/babylm-baseline-10m-gpt-bert-causal-focus>

^e <https://huggingface.co/BabyLM-community/babylm-baseline-10m-gpt-bert-mixed>

Table 5

Evaluation scores on BabyLM tasks (Excluding AoA) for dense and MoEP models. Percentage change indicates the relative improvement or decline in accuracy when replacing each dense baseline with its corresponding MoEP variant.

Model	Zero-shot Tasks						Finetuned Tasks							Macro
	BLiMP	EWOK	Entity	WUG	Comps	Reading	BoolQ	MNLI	MRPC	MultiRC	QQP	RTE	WSC	Avg
GPT-2	59.70	57.85	13.15	36.00	51.20	6.40	67.50	49.10	69.60	66.70	71.55	62.60	63.45	51.90
GPT-2-MoEP ^a	59.15	50.20	35.65	33.00	50.70	6.70	66.20	48.10	70.10	64.50	70.75	62.60	67.30	52.70
%-change	-0.90	-13.20	+170.10	-8.35	-1.00	+4.70	-1.90	-2.05	+0.7	-3.30	-1.10	0.00	+5.70	+1.55
Llama	62.20	49.40	40.20	36.00	51.65	5.85	67.40	48.35	70.10	66.10	70.50	53.95	53.85	51.95
Llama-MoEP ^b	59.30	50.00	26.00	36.00	51.40	6.25	66.80	48.90	69.60	66.30	72.55	60.45	63.45	52.10
%-change	-4.65	+1.2	-35.30	0.00	-0.50	+6.85	-1.20	+1.15	-0.70	+0.30	+2.90	+12.05	+17.85	+0.30
Gemma	59.65	49.80	17.50	43.00	51.80	6.85	67.05	49.35	71.10	65.70	71.80	55.40	65.40	51.90
Gemma-MoEP ^c	61.00	50.40	31.00	32.50	51.60	6.30	66.25	52.70	71.10	65.05	71.70	59.00	61.55	52.30
%-change	+1.20	+0.01	+77.15	-24.40	-0.40	-8.00	-1.20	+6.80	0.00	-1.00	-0.15	+6.50	-5.90	+0.80

^a <https://huggingface.co/Jtapsa/moep>

^b https://huggingface.co/Jtapsa/moep_llama

^c https://huggingface.co/Jtapsa/moep_gemma

particularly large variation across baseline-MoEP comparisons, indicating that MoEP exhibits notably different learning behavior on this task. These results motivate the routing analysis in Section 4.3, where we examine whether expert utilization and routing balance help explain the observed task-level differences.

4.2. Training dynamics

This section examines the evolution of average zero-shot evaluation accuracy across training checkpoints for all models. Our analysis focuses on how dense and sparse (MoEP) architectures progress under identical data and optimization settings. Fig. 3 summarizes checkpoint-wise performance for both model families.

4.2.1. MoEP models

Fig. 3a reports the average zero-shot evaluation accuracy for the MoEP models. All three variants showed broadly similar learning dynamics with only small variations, even though they reach their peak performance at different checkpoints. MoEP-GPT-2 indicates an early training gain, peaking at around 36.0% accuracy at the 30M checkpoint. After this point, performance begins to plateau following a noticeable drop, suggesting slight overfitting as the model continues to specialize. Beneath the average accuracy trends, this pattern is typically reflected

by improvements in tasks such as **BLiMP** and declines in tasks like **Entity Tracking**. MoEP-Gemma begins with a similarly sharp increase as the other MoEP models but experiences a more abrupt decline thereafter, eventually peaking at the lowest accuracy around the 50M checkpoint before stabilizing after a slight drop. The MoEP-LLaMA variant exhibits a learning curve comparable to MoEP-Gemma, but with a delayed rise instead of an early drop. It reaches its peak performance more than twice as late as MoEP-GPT-2 at 70M checkpoint, while achieving a comparable accuracy level.

Despite early-stage fluctuations, the MoEP variants exhibit more consistent late-stage learning behavior compared to their dense baselines, which tend to diverge in opposite directions. This suggests that sparse routing promotes a more generalizable learning pattern that is less dependent on layer-level architectural differences as compared to dense models.

4.2.2. Baselines

The dense baselines (GPT-2, LLaMA, and Gemma) exhibit more general and earlier learning dynamics (Fig. 3). Especially, both Gemma and GPT-2 reach their peak overall accuracy after the 30M checkpoint, while their accuracies differ by nearly 2%. The GPT-2 baseline diverges notably from its MoEP-GPT-2 counterpart in middle stage training, where the latter achieves a higher and more sustained performance peak.

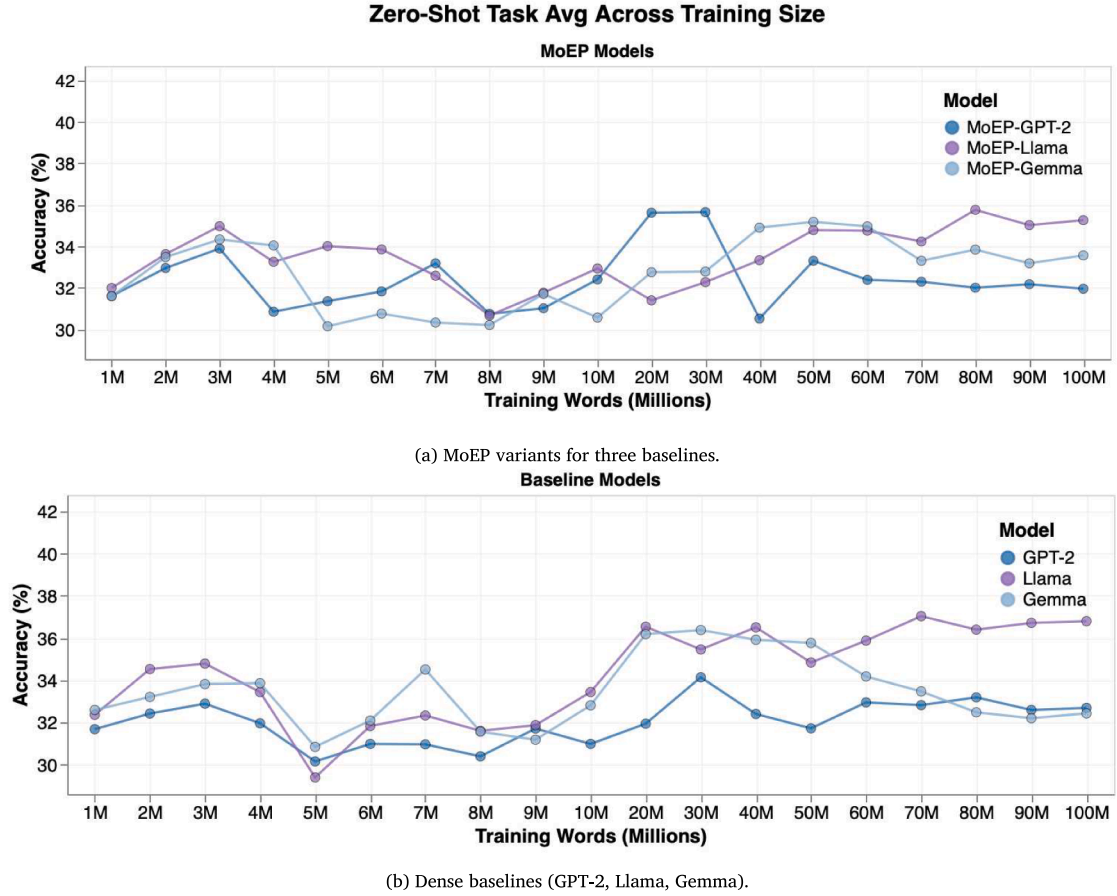


Fig. 3. Average zero-shot evaluation accuracy across model training checkpoints. Fig. 3a shows MoEP variants for three baselines, and the Fig. 3b shows dense baselines. Results illustrate overall learning trends and performance differences between architectures in dense and sparse setups.

Compared to their sparse counterparts *MoEP-Gemma* and *MoEP-LLaMA*, their corresponding dense baselines *Gemma* and *LLaMA* displays relatively faster learning and higher peak performance, which can be seen as a sharper increase and more stable curve in Fig. 3b. In those cases, the use of *MoEP* appears to moderate the learning process, leading to a more balanced overall training dynamic. In contrast, after the middle-stage training peak, the dense *LLaMA* continues to improve steadily, reaching its maximum performance at the 70M checkpoint, while *Gemma* follows the opposite trajectory with a gradual decrease.

Across all baselines, learning evolves smoothly without sharp changes, suggesting that dense pathways support more stable optimization but may also limit the models flexibility to learn. In contrast, *MoEP* architectures reduce this limitation, finding a balance between efficiency and adaptability.

4.3. Routing dynamics

4.3.1. Expert level

To better understand how sparse pathways are utilized during inference, we analyzed the layer-wise expert routing behavior of all *MoEP* variants using BLiMP evaluation samples. Fig. 4 visualizes the mean routing probabilities for each expert under both Top-1 and Top-2 configurations. Although the model employs top-2 routing, conducting both analysis reveals differences in how each model learns to distribute expert usage. The top-1 routing, plotted in Fig. 4a, illustrates how experts would divide if only one expert were activated during inference. Notably, *MoEP-LLaMA* exhibits evenly uniform routing probabilities of

25% across all experts, while *MoEP-GPT-2* shows strong imbalance, with two experts dominating and Expert 4 receiving less than 10% probability. *MoEP-Gemma* falls between these two extremes, displaying a partially balanced distribution where two experts reach approximately 25% probability each.

The top-2 expert routing, shown in Fig. 4b, differs notably from top-1 frequencies. While *MoEP-LLaMA* previously demonstrated an even distribution, it now shows the strongest imbalance, with Expert 1 dominating nearly 40% of the total probability. Likewise, *MoEP-Gemma* demonstrates a stable routing pattern, where Experts 1 and 4 maintain a balanced 25% distribution, and the earlier imbalance between Experts 2 and 3 becomes more even. *MoEP-GPT-2*, which was the most unbalanced in the top-1 configuration, smooths out substantially in top-2 routing, with Expert 4 increasing to 25%. In particular, all three models share a common 25% top-2 routing probability for Expert 4.

4.3.2. Layer level

Analyzing the mean routing probabilities across experts revealed several interesting aspects of model behavior. To gain more detailed insight, we further expand the analysis to the layer level. Fig. 5 visualizes the mean routing probability scores for each expert across layers, where color intensity represents the average activation frequency, and the red borders highlight the top-2 most active experts per layer.

MoEP-GPT-2 exhibits a moderately balanced yet structured routing pattern. Expert 1 dominates the early and middle layers, while Expert 2 is more active in the initial and final layers. Although either Expert 1 or Expert 2 is activated in every layer, their activity alternates, overlapping in only three out of twelve layers. Expert 3 shows consistent con-

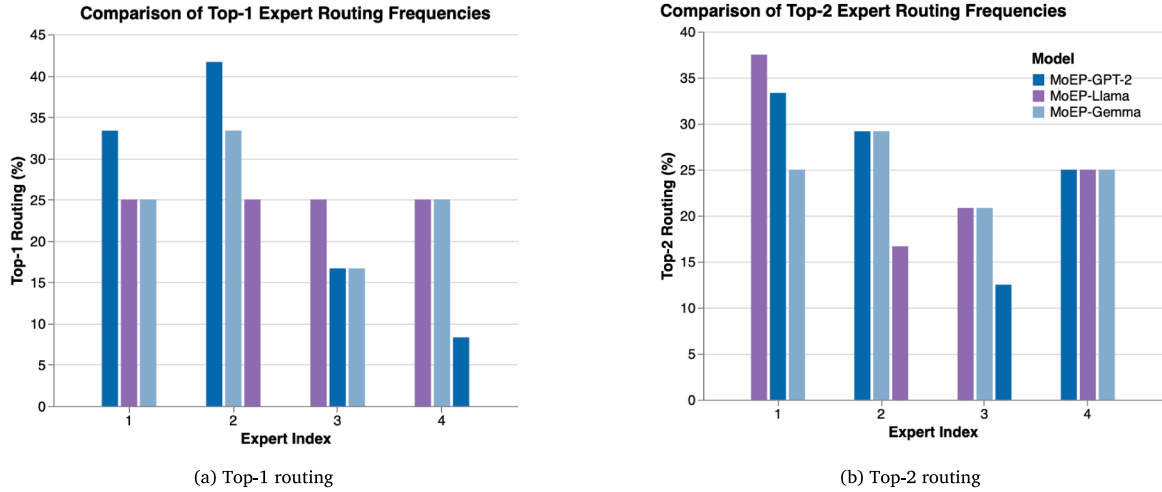


Fig. 4. Task- and layer-wise routing dynamics on the BLiMP benchmark with top-1 and top-2 routing.

tributions in the early layers, whereas Expert 4 has a weak and sparse influence throughout the network.

In contrast, *MoEP-LLaMA* exhibits a more asymmetric routing profile, with Expert 1 receiving the largest share, nearly 40% of the total routing activity. The remaining experts are distributed sparsely, showing less consistent activation patterns than in *MoEP-GPT-2*. However, several layers display more evenly distributed expert probabilities compared to the other models, suggesting a less specialized layer-level routing mechanism. This behavior may be related to the longer training duration of *MoEP-LLaMA* relative to the other variants.

Expert 4 reveals a clear connection between *MoEP-Gemma* and *MoEP-LLaMA*, as it exhibits an almost identical activation pattern in both models. This is noteworthy, since the overall expert activation behavior differs substantially between them. Moreover, *MoEP-Gemma* displays a nearly even expert distribution with some recurring activation patterns across layers. In comparison, *MoEP-LLaMA* shows more consistent patterns and clearer distinctions between active and inactive experts across its MoE layers.

Overall, the results suggest that routing balance correlates with evaluation performance in the corresponding analysis and test setups. *MoEP-Gemma* exhibits a more balanced and consistent routing distribution with stronger expert dominance, which aligns with its highest score in the analyzed *BLiMP* task. The duration of training may also influence routing behavior, where models peaking too early or too late tend to develop more imbalanced distributions, while the mid-point peak of *MoEP-Gemma* may partly explain its more stable and balanced routing. However, this observation would require further analysis to confirm.

5. Scalability stress test with Pythia-1B

To further examine whether the small-scale BabyLM results transfer to a notably larger scale, we conducted an additional scalability stress test using a Pythia 1B-parameter model (Biderman et al., 2023) as the dense baseline and created a corresponding MoEP-Pythia variant. Rather than replacing the BabyLM evaluation, the purpose of this experiment is to test whether the same layer-level routing design preserves its relative behavior when model size and training tokens are increased.

The MoEP-Pythia variant was trained using the OLMo-core training pipeline with the same training data, data order, and pre-training hyperparameters as the dense Pythia baseline. The only differences are the MoEP-specific architectural and routing components, which are summarized in Table 6. As with GPT-2, the Pythia architecture is relatively old, but it was selected for this stress test because it is straightforward to reproduce within a reasonable training budget.

Table 6

Architectural comparison between the dense Pythia-1B reproduction and the MoEP-Pythia variant used in the scalability stress test.

Component	Dense Pythia-1B	MoEP-Pythia
Model dimension	$d_{\text{model}} = 2048$	$d_{\text{large}} = 2048$, $d_{\text{parallel}} = 1024$
Layer structure	16 dense Transformer layers	1 full-dimensional input layer, 14 routed parallel layers, and 1 full-dimensional output layer
Full-dimensional layers	All layers use $d = 2048$, $n_{\text{heads}} = 8$, head dimension = 256	Input and output layers use $d = 2048$, $n_{\text{heads}} = 8$, head dimension = 256
FFN size in full-dimensional layers	FFN hidden dimension = 8192	FFN hidden dimension = 8192
MoE shrink projection	Not applicable	Maps 2048 $\rightarrow$ 1024 using 4 linear experts with top- $k = 2$ routing
Parallel routed layers	Not applicable	14 routed layers with 4 parallel paths and path top- $k = 2$
Parallel-path configuration	Not applicable	Each path uses $d = 1024$, $n_{\text{heads}} = 4$, head dimension = 256, FFN hidden dimension = 4096
MoE grow projection	Not applicable	Maps 1024 $\rightarrow$ 2048 using 4 linear experts with top- $k = 2$ routing
Non-embedding parameters	805,736,448 total	822,968,384 total, 461,879,360 active per token (56.1%)

MoEP-Pythia was trained Pythia 1B matched conditions on the LUMI supercomputer using 32 computation nodes, where each node contains 4 MI250x GPUs (8 GCDs) and training took around 46 h i.e 5900 GPU/h (CSC, 0000). After training of MoEP-Pythia, we evaluated checkpoints from 10B to 300B training tokens, where the corresponding Pythia model was loaded from Hugging Face.⁶

Because these models are notably larger and trained on substantially more data than the BabyLM-scale models, they were evaluated with a separate downstream evaluation setup that is more appropriate for this scale. Both model checkpoints were evaluated using the OLMo-eval suite with 8 multiple-choice reasoning tasks: ARC-Challenge, ARC-Easy, COPA, CommonsenseQA, HellaSwag, PIQA, SocialIQA, and Wino-

⁶ <https://huggingface.co/EleutherAI/pythia-1b>

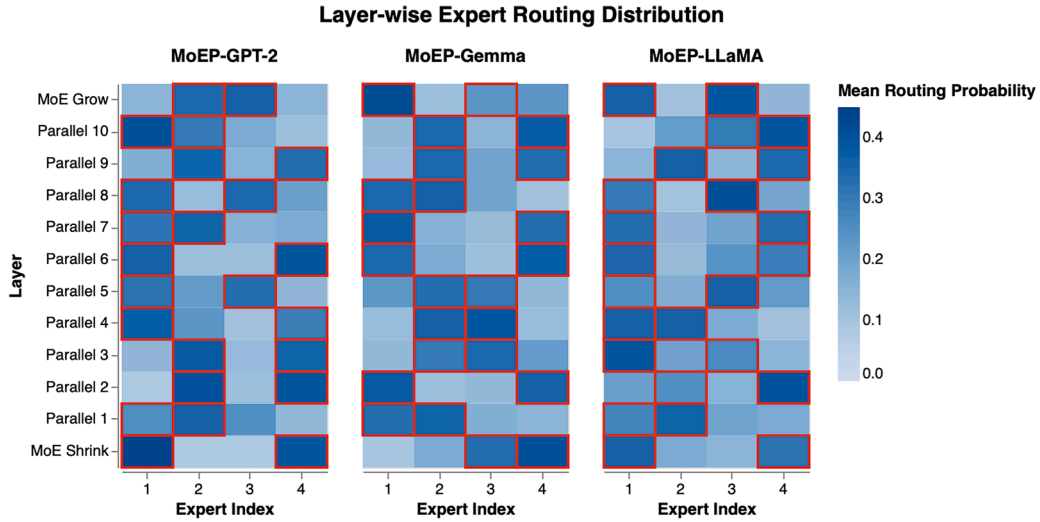


Fig. 5. Layer-wise expert routing probabilities across MoEP models. Color intensity indicates mean routing probability per expert per layer, and red borders highlight the top-2 most active experts.

Table 7

Average downstream performance difference between MoEP-Pythia and dense Pythia across training checkpoints. Positive values indicate higher average performance for MoEP-Pythia; negative values indicate lower average performance.

Training tokens	Pythia	MoEP-Pythia	Diff
10B	44.35	45.74	+1.39
25B	47.32	47.09	-0.23
50B	48.89	47.80	-1.09
75B	50.25	48.29	-1.96
100B	49.93	48.07	-1.86
125B	49.76	49.09	-0.67
150B	50.62	49.20	-1.42
175B	51.14	49.97	-1.17
200B	51.24	49.79	-1.45
225B	51.86	50.03	-1.83
250B	50.82	48.86	-1.96
275B	51.73	49.95	-1.78
300B	52.11	49.66	-2.45

grande. All of those tasks were evaluated in the zero-shot setting, denoted by the `rc_0shot` suffix in OLMo-eval.

The results in Table 7 show that MoEP-Pythia slightly outperforms the dense Pythia baseline at the earliest 10B training token checkpoint, with average scores of 45.74 and 44.35. However, this checkpoint corresponds to only 3.3% of the full 300B-token training run, and the initial advantage does not persist as the size increases. From 25B to 300B training tokens, MoEP-Pythia underperforms the dense baseline at every evaluated checkpoint. Across these later checkpoints, the average difference remains negative, ranging approximately from -0.23 to -2.45 points. This strongly indicates that the current MoEP design cannot sustain a performance advantage over the dense Pythia baseline at the 1B-parameter scale.

Our Pythia-scale experiment therefore reveals an important scalability limitation of the current MoEP configuration.

5.1. Computational efficiency

To further expand the Pythia-scale performance comparison, we also measured the computational efficiency of the dense Pythia-1B baseline and the corresponding MoEP-Pythia variant. Table 8 reports average training throughput and GPU memory usage for dense Pythia-1B baseline and MoEP-Pythia under the same 2 computation node / 16-GCD

Table 8

Computational efficiency comparison between dense Pythia-1B and MoEP-Pythia on 2 nodes / 16 GCDs. Relative speed is computed from average per-device throughput, using dense Pythia-1B as the reference.

Model	TPS/GCD	Relative speed	GPU mem
Pythia-1B	10,422	1.00×	10.82 (GiB)
MoEP-Pythia	9289	0.89×	17.13 (GiB)

training setup. In the current MoEP implementation, where parallel layer calculations are looped, MoEP-Pythia reaches 9289 tokens/s per GCD in comparison 10,422 tokens/s per GCD for dense Pythia-1B, which corresponding $0.89\times$ tput. MoEP-Pythia also uses notably higher active GPU memory, increasing it from 10.82 GiB to 17.13 GiB per GCD. These results show a problem in current MoEP-Pythia implementation, which does not provide practical wall-clock efficiency gains despite using fewer active non-embedding parameters per token and reduced dimensionality in parallel layers.

Although MoEP-Pythia reduces the number of active parameters, the layer-level routing used by MoEP causes additional overhead because it routes complete layers, including both attention and feed-forward computation. This differs from standard FFN-level MoE, where sparsity is included only in the feed-forward sublayers, while attention remains shared. Therefore, the Pythia-scale efficiency measurements with the current MoEP framework support the conclusion that active parameter count alone is insufficient to establish practical computational efficiency.

5.2. Why layer-level MoEP does not inherit FFN-level MoE efficiency?

The main challenge related to the efficiency of MoEP comes from routing complete Transformer blocks (layers) rather than only FFN experts. In standard FFN-level MoE, token routing can directly skip unselected FFN experts, because FFN computation is applied independently to each token. From this point of view, MoEP is different because each parallel path contains its own attention sublayer, where token representations are computed through interactions with all other tokens in the input sequence.

This means that even if a specific input token does not select a given parallel path, that token still needs to participate in the attention computation of that path if other tokens select it. This weakens the practical sparsity benefit of token-level path selection because without changing

the attention mechanism, the model cannot easily remove all unselected tokens from the attention computation.

To address this efficiency problem, a more specialized routing design would be required. One possibility is token-pruned attention, where tokens that do not select a specific path are removed from the corresponding attention computation path.

Several recent and earlier works have motivated this direction by implementing conditional computation. Mixture-of-Depths showed that Transformer layer computation can be dynamically allocated by allowing only a subset of selected tokens to attend in the self-attention and MLP computation at each layer (Raposo et al., 2024). Another possible solution to lighten attention computation is to follow the MoBA demonstrated style, where queries are routed only to selected key-value blocks rather than computing dense attention over the full context (Lu, 2025).

A complementary direction is batch-level or group-level routing, where all tokens in a batch or routing group are routed through the same layer-level paths. This idea is distantly related to MoCE-style grouped MoE designs, where sequence-level features are used to select an expert group before applying the actual token-level routing inside that group (Eo et al., 2025).

These designs could make active paths more hardware-efficient, but they would also change the current token-level MoEP formulation and may reduce routing flexibility and representational capacity.

6. Discussion and conclusion

6.1. Interpretation of training and routing dynamics

Training and routing pattern analysis revealed more insight about how learning progresses in both dense and sparse models. Across our dense baselines, learning evolved smoothly without sharp changes, suggesting that dense pathways support stable learning but also limit flexibility. In contrast, **MoEP** models learn slower and reach their peak later, while maintaining stable behavior in later stages of training (Fig. 3).

Among sparse variants, *MoEP-GPT-2* peaks early around 30M words and then plateaus after a rapid drop. *MoEP-Gemma* shows a sharp increase in early training, followed by a sudden drop before stabilizing near 50M, and *MoEP-LLaMA* reaches a similar accuracy but peaks much later, near 70M. These patterns show that baseline architectures have a strong affect both learning efficiency and routing balance, models peaking too early or too late tend to form more uneven expert distributions.

Later routing analysis on the *Blimp* task in Fig. 4 supports this view. Although the model uses top-2 routing in training and inference, comparing top-1 and top-2 setups revealed how experts share load across layers. Specifically, *MoEP-GPT-2* alternates between Experts 1 and 2, *MoEP-LLaMA* seems to centralize routing on one dominant expert and *MoEP-Gemma* distributes activation more evenly. These differences reflect how sparsity interacts with baseline design and training progression rather than directly correlates overall accuracy.

6.2. Architectural implications

The routing results, combined with the learning analysis, showed how architecture and training length shape model behavior. Among the dense baselines, *Gemma* and *LLaMA* follow very similar training curves compared to *GPT-2*, but around the midpoint their performance starts to diverge. This difference is mainly driven by the much lower score of *LLaMA* in the *Entity Tracking* task, where *Gemma* remains closer to *GPT-2*. In the routing analysis, both *GPT-2* and *Gemma* also show a more balanced expert distribution than *LLaMA*. This suggests that the layer-level FeedForward network architecture may influence how evenly routing probabilities are distributed. While their details differ, *Gemma* and *GPT-2* both use the GELU activation function, whereas *LLaMA* uses SiLU, which may contribute to these differences.

Notably, the overall accuracy of the dense baselines in Section 4.1 is nearly identical, with only a 0.05-point difference between them. Each

model shows specialization on certain tasks, while performance on other tasks remains very similar, and paired t-tests do not indicate a statistically significant gap. This suggests that, under the BabyLM strict-small setup, architectural differences between dense baselines mainly lead to task-specific variation rather than more systematic average performance differences.

Larger-scale comparison in Section 5 shows that the effectiveness of MoEP can notably decrease at larger model and training scales, which may not be explained only by baseline architectural differences. Instead, it may reflect the increasing gap between the dense model dimension and the reduced parallel-path dimension, which becomes more prominent in large scale. As the model scale increases, reduced dimensional in routed paths may no longer provide enough representational capacity to match the dense baseline. Increasing the parallel-path dimension more closer to dense or the number of routed paths could improve model capacity, but same time it would also increase computational and memory overhead because MoEP routes complete Transformer blocks, including attention.

6.3. Relation to prior work

Our observations align to some extent with the findings from *PapaFormer* (Tapaninaho & Oussalah, 2025), which reported that more uniformly shared expert routing correlates with improved model performance and stability. In that study, the expert load imbalance was identified as a key factor influencing accuracy across evaluation tasks. Our further analysis partly confirms this: when examining top-1 routing, having one dominant expert can be beneficial, but the top-2 routing results reveal that a more balanced distribution across experts correlates with better overall performance. This inconsistency may be explained by the fact that *PapaFormer* activates only one expert during inference, which can likely lead to stronger dominance of a single expert.

Together, these results support the conjecture that having stronger experts, with dominating probability scores, combined with balance across all experts, is linked to higher overall accuracy.

6.4. Limitations

In the main experiments, all *MoEP* models were trained under a constrained computational budget and with the BabyLM strict-small corpus. Therefore, the main results should be interpreted within a small-model and low-resource setting. Within the BabyLM setup, where the training corpus and learned patterns are relatively constrained, parallel blocks with reduced dimension can still capture useful representations. However, with much larger and more complex training data, these reduced-dimensional paths may no longer provide sufficient representational capacity, which can weaken notably MoEP performance relative to dense baselines.

This limitation is supported by the additional Pythia-1B scalability experiment, where the current MoEP-Pythia configuration cannot match the performance of the dense Pythia baseline. This suggests that the small-scale gains observed under BabyLM do not transfer directly to 1B-class training in the current setup. Additionally, MoEP applies routing to complete Transformer blocks, including both attention and feed-forward computation, rather than only to FFN experts. As a result, in the current implementation, MoEP does not inherit the same sparsity-efficiency or wall-clock efficiency trade-off as standard FFN-level MoE architectures.

A further limitation is that this study does not include a full ablation of the different parallel hidden dimensionalities or the number of routed experts. These hyperparameters directly control the trade-off between representational capacity and computational cost. Due to the resource requirements of training multiple MoEP variants, we evaluate one fixed large-scale MoEP configuration in addition to the smaller-scale architectural comparisons, rather than exploring the full MoEP design space. Future work should therefore investigate at which model scale the MoEP

advantage disappears, and whether this transition depends on expert count or parallel hidden dimension.

6.5. Conclusion

In overall, *MoEP* provides a conditional layer-level routing framework with parallel paths for studying modular sparsity in decoder-only Transformer architectures. Under the constrained BabyLM strict-small protocol, all *MoEP* variants show small average improvements over dense baselines with stronger gains on selected tasks, but those results are not uniform across all architectures or evaluations.

The additional Pythia-scale analysis further shows that the current *MoEP* configuration does not directly transfer to 1B-class training. In conclusion, *MoEP* is best interpreted as a useful framework for analyzing layer-level conditional computation at small model scale, while its scalability and efficiency require further architectural development.

CRedit authorship contribution statement

Joonas Tapaninaho: Writing – original draft, Software, Methodology, Investigation, Formal analysis, Conceptualization; **Mourad Oussalah:** Writing – review & editing, Supervision, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is partly supported by the EU Chanse DiGeMERGE project and University of Oulu funding, which are gratefully acknowledged. The authors acknowledge CSC-IT Center for Science, Finland, for computational resources. The large-scale experiments were mainly carried out using resources allocated under Project 462001445.

References

- Bae, S., Kim, Y., & Bayet, R., et al. (2025). Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *Advances in neural information processing systems*. To appear <https://arxiv.org/abs/2507.10524>.
- Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O'Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., Skowron, A., Sutawika, L., & Van Der Wal, O. (2023). Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th international conference on machine learning* (pp. 2397–2430). PMLR (vol. 202). Proceedings of Machine Learning Research. <https://proceedings.mlr.press/v202/biderman23a.html>.
- Charpentier, L., Choshen, L., & Cotterell, R., et al. (2025). BabyLM turns 3: Call for papers for the 2025 babyLM workshop. [arXiv:2502.10645](https://arxiv.org/abs/2502.10645).
- Charpentier, L. G. G., & Samuel, D. (2024). GPT or BERT: Why not both? In *Proceedings of the 2nd babyLM challenge workshop*. [arXiv:2410.24159](https://arxiv.org/abs/2410.24159).
- Chen, K., Xu, L., & Chi, H. (1999). Improved learning algorithms for mixture of experts in multiclass classification. *Neural Networks*, 12(9), 1229–1252. [https://doi.org/10.1016/S0893-6080\(99\)00043-X](https://doi.org/10.1016/S0893-6080(99)00043-X).
- Chowdhery, et al. (2023). PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24, 1–113. <https://www.jmlr.org/papers/v24/22-1144.html>.
- CSC, LUMI Supercomputer. Lumi supercomputer. <https://lumi-supercomputer.eu/>. Accessed: 2026-06-16.
- CSC, Puhti Supercomputer. Puhti supercomputer. <https://docs.csc.fi/computing/systems-puhti/>. Accessed: 2025-08-06.
- DeepSeek-AI (2024). Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. [arXiv:2405.04344](https://arxiv.org/abs/2405.04344).
- DeepSeek-AI (2025). Deepseek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. [arXiv:2501.12948](https://arxiv.org/abs/2501.12948).
- Dou, S., Zhou, E., Liu, Y., Gao, S., Shen, W., Xiong, L., Zhou, Y., Wang, X., Xi, Z., Fan, X., Pu, S., Zhu, J., Zheng, R., Gui, T., Zhang, Q., & Huang, X. (2024). LoRAMoE: Alleviate world knowledge forgetting in large language models via MoE-style plugin. <https://aclanthology.org/2024.acl-long.106/>. <https://doi.org/10.18653/v1/2024.acl-long.106>.
- Du, N., Huang, Y., Dai, A. M., Tong, S., Lepikhin, D., Xu, Y., Krikun, M., Zhou, Y., Wei Yu, A., Firat, O., Zoph, B., Fedus, L., Bosma, M., Zhou, Z., Wang, T., Emma Wang, Y., Webster, K., Pellat, M., Robinson, K., Meier-Hellstern, K., Duke, T., Dixon, L., Zhang, K., Le, Q. V., Wu, Y., Chen, Z., & Cui, C. (2022). GLaM: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the 39th international conference on machine learning* (pp. 5547–5569). (vol. 162). Proceedings of Machine Learning Research. <https://arxiv.org/abs/2112.06905>.
- EO, S., Lee, J. J., Park, C., & Lim, H. (2025). Mixture-of-clustered-experts: Advancing expert specialization and generalization in instruction tuning. In *Proceedings of the 2025 conference on empirical methods in natural language processing* (pp. 14201–14212). Suzhou, China: Association for Computational Linguistics. <https://aclanthology.org/2025.emnlp-main.718/>. <https://doi.org/10.18653/v1/2025.emnlp-main.718>.
- Fedus, W., Zoph, B., & Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120), 1–39. <https://www.jmlr.org/papers/v23/21-0998.html>.
- Fu, et al. (2024). Moa: Mixture of sparse attention for automatic large language model compression. [arXiv:2406.14909](https://arxiv.org/abs/2406.14909).
- Grattafiori, et al. (2024). The Llama 3 herd of models. [arXiv:2407.21783](https://arxiv.org/abs/2407.21783).
- He, S., Dong, D., Ding, L., & Li, A. (2025). Towards efficient mixture of experts: A holistic study of compression techniques. *Transactions on Machine Learning Research*. <https://openreview.net/forum?id=HTpM0l6xSI>.
- Hu, E., Shen, Y., & Wallis, P., et al. (2022). LoRA: Low-rank adaptation of large language models. In *Proceedings of the international conference on learning representations*. <https://arxiv.org/abs/2106.09685>.
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J., & Hinton, G. E. (1991). Adaptive mixtures of local experts. *Neural Computation*, 3(1), 79–87. <https://doi.org/10.1162/neco.1991.3.1.79>.
- Jiang, et al. (2024). Mixtral of experts. [arXiv:2401.04088](https://arxiv.org/abs/2401.04088).
- Jiang, W., & Tanner, M. (1999). Hierarchical mixtures-of-experts for exponential family regression models: Approximation and maximum likelihood estimation. *The Annals of Statistics*, 27(3), 987–1011. <https://doi.org/10.1214/aos/1018031265>.
- Jordan, M. I., & Jacobs, R. A. (1994). Hierarchical mixtures of experts and the EM algorithm. *Neural Computation*, 6(2), 181–214. <https://doi.org/10.1162/neco.1994.6.2.181>.
- Jordan, M. I., & Xu, L. (1995). Convergence results for the EM approach to mixtures of experts architectures. *Neural Networks*, 8(9), 1409–1431. [https://doi.org/10.1016/0893-6080\(95\)00014-3](https://doi.org/10.1016/0893-6080(95)00014-3).
- Lu, E., et al. (2025). MoBA: Mixture of block attention for long-context LLMs. <https://arxiv.org/abs/2502.13189>.
- Meta-AI (2025). The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>. Accessed: 2025-09-20.
- Muennighoff, et al. (2025). OLMoE: Open mixture-of-experts language models. In *Proceedings of the international conference on learning representations*. <https://arxiv.org/abs/2409.02060>.
- OpenAI (2025). GPT-OSS-120b & GPT-OSS-20b model card. [arXiv:2508.10925](https://arxiv.org/abs/2508.10925).
- Pan, B., Shen, Y., Liu, H., Mishra, M., Zhang, G., Oliva, A., Raffel, C., & Panda, R. (2024). Dense training, sparse inference: Rethinking training of mixture-of-experts language models. [arXiv:2404.05567](https://arxiv.org/abs/2404.05567).
- Peng, et al. (2022). Branchformer: Parallel MLP-attention architectures to capture local and global context for speech recognition and understanding. In *Proceedings of the 39th international conference on machine learning* (pp. 17627–17643). (vol. 162). Proceedings of Machine Learning Research. <https://arxiv.org/abs/2207.02971>.
- Raposo, D., Ritter, S., Richards, B., Lillicrap, T., Humphreys, P. C., & Santoro, A. (2024). Mixture-of-depths: Dynamically allocating compute in transformer-based language models. <https://arxiv.org/abs/2404.02258>.
- Shen, Y., Guo, Z., Cai, T., & Qin, Z. (2024). JetMoE: Reaching Llama2 performance with 0.1m dollars. [arXiv:2404.07413](https://arxiv.org/abs/2404.07413).
- Tapaninaho, J. (2025). MoEP: Modular expert paths for sample-efficient language modeling. In *Proceedings of the first babyLM workshop* (pp. 540–547). Suzhou, China: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.babyLM-main.39>.
- Tapaninaho, J., & Oussalah, M. (2025). Papaformer: Language model from pre-trained parallel paths. [arXiv:2508.00544](https://arxiv.org/abs/2508.00544).
- Team, G. (2024a). Gemma 2: Improving open language models at a practical size. [arXiv:2408.00118](https://arxiv.org/abs/2408.00118).
- Team, G. (2024b). Gemma: Open models based on gemini research and technology. [arXiv:2403.08295](https://arxiv.org/abs/2403.08295).
- Touvron, et al. (2023a). LLaMA 2: Open foundation and fine-tuned chat models. [arXiv:2307.09288](https://arxiv.org/abs/2307.09288).
- Touvron, et al. (2023b). LLaMA: Open and efficient foundation language models. <https://arxiv.org/abs/2302.13971>.
- Vaswani, A., Shazeer, N., & Parmar, N., et al. (2017). Attention is all you need. In *Advances in neural information processing systems*. (vol. 30). <https://arxiv.org/abs/1706.03762>.
- Wang, et al. (2022). AdaMix: Mixture-of-adaptations for parameter-efficient model tuning. In *Proceedings of the 2022 conference on empirical methods in natural language processing* (pp. 5744–5760). <https://arxiv.org/abs/2205.12410>.
- Wu, X., Huang, S., & Wei, F. (2024). Mixture of LoRA experts. In *Proceedings of the international conference on learning representations*. <https://arxiv.org/abs/2404.13628>.
- Zadouri, T., Üstün, A., Ahmadian, A., Ermis, B., Locatelli, A., & Hooker, S. (2024). Pushing mixture of experts to the limit: Extremely parameter efficient MoE for instruction tuning. In *Proceedings of the international conference on learning representations*. <https://arxiv.org/abs/2309.05444>.
- Zhang, Z., Xia, Y., Wang, H., Yang, D., Hu, C., Zhou, X., & Cheng, D. (2024). MpMoE: Memory efficient moe for pre-trained models with adaptive pipeline parallelism. <https://doi.org/10.1109/TPDS.2024.3385639>.

Zhou, et al. (2022). Mixture-of-experts with expert choice routing. In *Advances in neural information processing systems*. (vol. 35). <https://arxiv.org/abs/2202.09368>.

Zhu, Y., Wichers, N., Lin, C.-C., Wang, X., Chen, T., Shu, L., Lu, H., Liu, C., Luo, L., Chen, J., & Meng, L. (2023). SiRA: Sparse mixture of low rank adaptation. [arXiv:2311.09179](https://arxiv.org/abs/2311.09179).

Zuo, S., Liu, X., Jiao, J., Kim, Y. J., Hassan, H., Zhang, R., Zhao, T., & Gao, J. (2022). Taming sparsely activated transformer with stochastic experts. In *Proceedings of the international conference on learning representations*. <https://arxiv.org/abs/2110.04260>.